

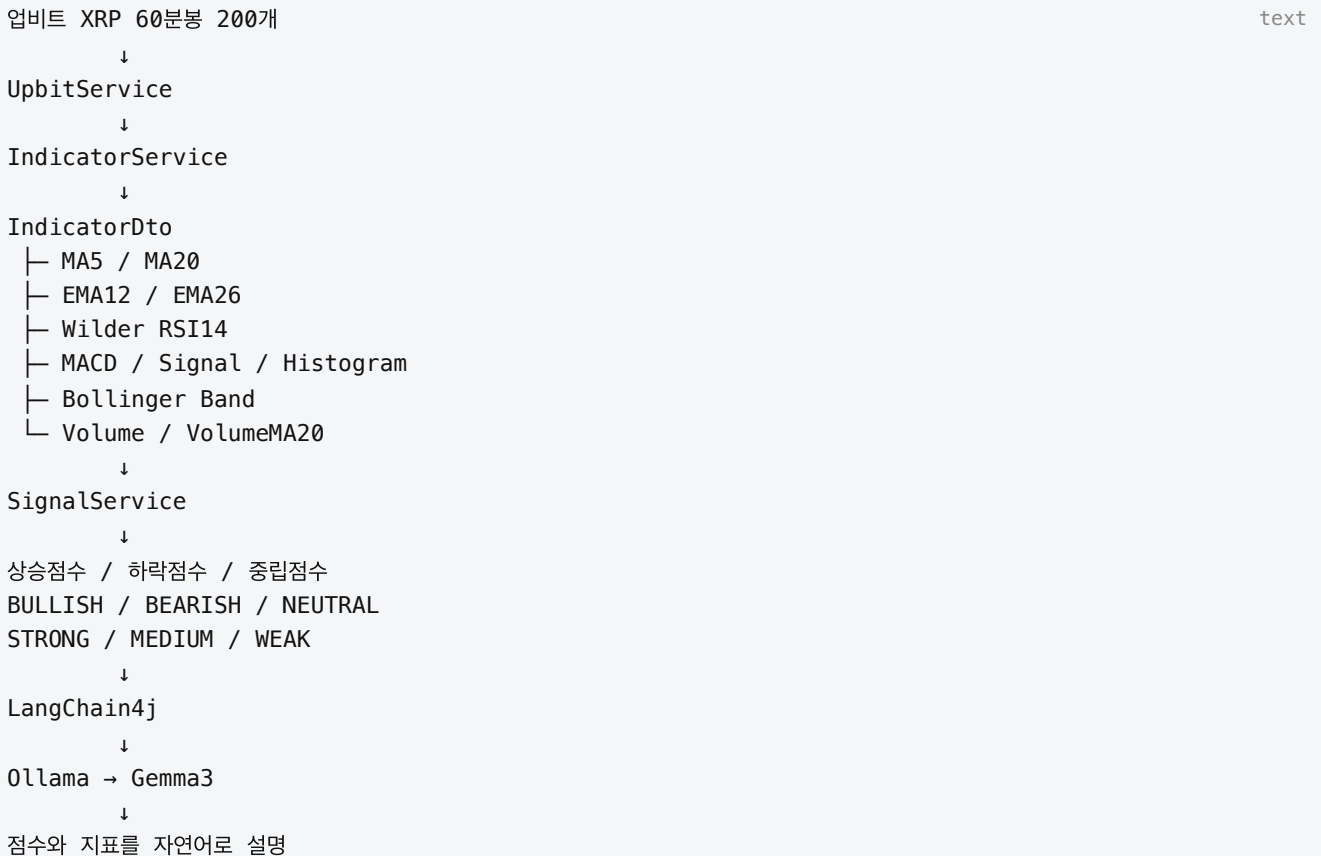
## Spring Boot Java + Kotlin 신호점수 실전 예제

IndicatorDto → SignalService → 상승·하락 점수 → LangChain4j → Gemma3

0llama: http://localhost:11434

yaml

### 1. 전체 구조



역할 분리: IndicatorService = 지표 계산, SignalService = 규칙·점수 계산, Gemma3 = 계산 결과의 자연어 해석.

### 2. 학습용 점수 규칙

| 조건            | 상승점수 | 하락점수 |
|---------------|------|------|
| MA5 > MA20    | +20  | -    |
| MA5 < MA20    | -    | +20  |
| EMA12 > EMA26 | +20  | -    |
| EMA12 < EMA26 | -    | +20  |
| RSI 50~70     | +15  | -    |
| RSI 30~50     | -    | +10  |
| RSI ≥ 70      | -    | +10  |

| 조건                           | 상승점수      | 하락점수      |
|------------------------------|-----------|-----------|
| RSI $\leq$ 30                | +10       | -         |
| MACD Histogram > 0           | +20       | -         |
| MACD Histogram < 0           | -         | +20       |
| 가격 > BB Middle               | +10       | -         |
| 가격 < BB Middle               | -         | +10       |
| 거래량 $\geq$ MA20 $\times$ 1.2 | 우세 방향 +15 | 우세 방향 +15 |

주의: 이 점수는 상승·하락 확률이 아니라 학습용 규칙의 충족도입니다.

### 3. SignalDto

#### Java - SignalDto.java

```
public record SignalDto(
    int bullishScore,
    int bearishScore,
    int neutralScore,
    String signal,
    String strength,
    String reasons
) {}
```

text

#### Kotlin - SignalDto.kt

```
data class SignalDto(
    val bullishScore: Int,
    val bearishScore: Int,
    val neutralScore: Int,
    val signal: String,
    val strength: String,
    val reasons: String
)
```

kotlin

## 4. Java - SignalService

java

```
@Service
public class SignalService {

    public SignalDto calculate(IndicatorDto d) {

        int bullish = 0;
        int bearish = 0;
        List<String> reasons = new ArrayList<>();

        // 1. MA
        if (d.ma5() > d.ma20()) {
            bullish += 20;
            reasons.add("MA5가 MA20보다 높음");
        } else if (d.ma5() < d.ma20()) {
            bearish += 20;
            reasons.add("MA5가 MA20보다 낮음");
        }

        // 2. EMA
        if (d.ema12() > d.ema26()) {
            bullish += 20;
            reasons.add("EMA12가 EMA26보다 높음");
        } else if (d.ema12() < d.ema26()) {
            bearish += 20;
            reasons.add("EMA12가 EMA26보다 낮음");
        }

        // 3. RSI
        if (d.rsi14() >= 50 && d.rsi14() < 70) {
            bullish += 15;
            reasons.add("RSI가 상승 우위 구간");
        } else if (d.rsi14() >= 30 && d.rsi14() < 50) {
            bearish += 10;
            reasons.add("RSI가 약세 구간");
        } else if (d.rsi14() >= 70) {
            bearish += 10;
            reasons.add("RSI 과매수 주의");
        } else if (d.rsi14() <= 30) {
            bullish += 10;
            reasons.add("RSI 과매도 반등 가능성");
        }

        // 4. MACD
        if (d.macdHistogram() > 0) {
            bullish += 20;
            reasons.add("MACD Histogram 양수");
        } else if (d.macdHistogram() < 0) {
            bearish += 20;
            reasons.add("MACD Histogram 음수");
        }

        // 5. Bollinger
        if (d.price() > d.bollingerMiddle()) {
            bullish += 10;
            reasons.add("가격이 볼린저 중앙선 위");
        }
    }
}
```

```

    } else {
        bearish += 10;
        reasons.add("가격이布林저 중앙선 아래");
    }

    // 6. Volume
    double volumeRatio = d.volumeMa20() == 0
        ? 0 : d.volume() / d.volumeMa20();

    if (volumeRatio >= 1.2) {
        if (bullish > bearish) {
            bullish += 15;
            reasons.add("상승 조건에서 거래량 증가");
        } else if (bearish > bullish) {
            bearish += 15;
            reasons.add("하락 조건에서 거래량 증가");
        }
    }

    bullish = Math.min(bullish, 100);
    bearish = Math.min(bearish, 100);

    int neutral = Math.max(
        0, 100 - Math.max(bullish, bearish));

    String signal;
    if (bullish - bearish >= 20) {
        signal = "BULLISH";
    } else if (bearish - bullish >= 20) {
        signal = "BEARISH";
    } else {
        signal = "NEUTRAL";
    }

    int difference = Math.abs(bullish - bearish);

    String strength;
    if (difference >= 50) {
        strength = "STRONG";
    } else if (difference >= 25) {
        strength = "MEDIUM";
    } else {
        strength = "WEAK";
    }

    return new SignalDto(
        bullish, bearish, neutral,
        signal, strength,
        String.join(", ", reasons));
}
}

```

## 5. Kotlin - SignalService

kotlin

```
@Service
class SignalService {

    fun calculate(d: IndicatorDto): SignalDto {

        var bullish = 0
        var bearish = 0
        val reasons = mutableListOf<String>()

        // 1. MA
        if (d.ma5 > d.ma20) {
            bullish += 20
            reasons += "MA5가 MA20보다 높음"
        } else if (d.ma5 < d.ma20) {
            bearish += 20
            reasons += "MA5가 MA20보다 낮음"
        }

        // 2. EMA
        if (d.ema12 > d.ema26) {
            bullish += 20
            reasons += "EMA12가 EMA26보다 높음"
        } else if (d.ema12 < d.ema26) {
            bearish += 20
            reasons += "EMA12가 EMA26보다 낮음"
        }

        // 3. RSI
        when {
            d.rsi14 >= 50.0 && d.rsi14 < 70.0 -> {
                bullish += 15
                reasons += "RSI 상승 우위 구간"
            }
            d.rsi14 >= 30.0 && d.rsi14 < 50.0 -> {
                bearish += 10
                reasons += "RSI 약세 구간"
            }
            d.rsi14 >= 70.0 -> {
                bearish += 10
                reasons += "RSI 과매수 주의"
            }
            d.rsi14 <= 30.0 -> {
                bullish += 10
                reasons += "RSI 과매도 반등 가능성"
            }
        }

        // 4. MACD
        if (d.macdHistogram > 0) {
            bullish += 20
            reasons += "MACD Histogram 양수"
        } else if (d.macdHistogram < 0) {
            bearish += 20
            reasons += "MACD Histogram 음수"
        }
    }
}
```

```

// 5. Bollinger
if (d.price > d.bollingerMiddle) {
    bullish += 10
    reasons += "가격이布林저 중앙선 위"
} else {
    bearish += 10
    reasons += "가격이布林저 중앙선 아래"
}

// 6. Volume
val volumeRatio =
    if (d.volumeMa20 == 0.0) 0.0
    else d.volume / d.volumeMa20

if (volumeRatio >= 1.2) {
    if (bullish > bearish) {
        bullish += 15
        reasons += "상승 조건에서 거래량 증가"
    } else if (bearish > bullish) {
        bearish += 15
        reasons += "하락 조건에서 거래량 증가"
    }
}

bullish = bullish.coerceAtMost(100)
bearish = bearish.coerceAtMost(100)

val neutral =
    (100 - maxOf(bullish, bearish)).coerceAtLeast(0)

val signal = when {
    bullish - bearish >= 20 -> "BULLISH"
    bearish - bullish >= 20 -> "BEARISH"
    else -> "NEUTRAL"
}

val difference =
    kotlin.math.abs(bullish - bearish)

val strength = when {
    difference >= 50 -> "STRONG"
    difference >= 25 -> "MEDIUM"
    else -> "WEAK"
}

return SignalDto(
    bullishScore = bullish,
    bearishScore = bearish,
    neutralScore = neutral,
    signal = signal,
    strength = strength,
    reasons = reasons.joinToString(", ")
)
}
}

```

## 6. 계산 예시

입력 지표 예:

현재 XRP = 4,210

MA5 = 4,190 / MA20 = 4,130

EMA12 = 4,185 / EMA26 = 4,145

RSI14 = 62

MACD Histogram = +12

Bollinger Middle = 4,130

현재 거래량 = 3,000,000

Volume MA20 = 2,000,000

점수:

MA +20

EMA +20

RSI +15

MACD +20

Bollinger +10

거래량 증가 +15

-----  
상승점수 = 100

text

100점은 '상승확률 100%'가 아니라 정의한 규칙을 강하게 충족했다는 뜻입니다.

## 7. Java - Gemma3 분석 Service

java

```
@Service
public class XrpAiService {

    private final ChatModel chatModel;

    public XrpAiService(ChatModel chatModel) {
        this.chatModel = chatModel;
    }

    public String analyze(IndicatorDto i, SignalDto s) {

        String prompt = ""
            + "당신은 XRP 기술적 분석 설명 AI입니다."

            + "[기술지표]"
            + "현재가격: %.2f"
            + "MA5: %.2f"
            + "MA20: %.2f"
            + "EMA12: %.2f"
            + "EMA26: %.2f"
            + "Wilder RSI14: %.2f"
            + "MACD: %.4f"
            + "Signal: %.4f"
            + "Histogram: %.4f"
            + "Bollinger Upper: %.2f"
            + "Bollinger Middle: %.2f"
            + "Bollinger Lower: %.2f"

            + "[Spring Boot 신호 계산]"
            + "상승점수: %d"
            + "하락점수: %d"
            + "중립점수: %d"
            + "판정: %s"
            + "강도: %s"
            + "계산 근거: %s"

            + "계산 결과를 변경하지 말고 이해하기 쉽게 설명하세요."

            + "현재 추세:"
            + "지표 해석:"
            + "상승 근거:"
            + "하락 위험:"
            + "거래량 해석:"
            + "종합 의견:"
            + "주의사항:"

            + "점수를 상승 확률로 표현하지 마세요."
            + "확정적인 투자 조언을 하지 마세요."
            + """.formatted(
                i.price(), i.ma5(), i.ma20(),
                i.ema12(), i.ema26(), i.rsi14(),
                i.macd(), i.macdSignal(), i.macdHistogram(),
                i.bollingerUpper(), i.bollingerMiddle(), i.bollingerLower(),
                s.bullishScore(), s.bearishScore(), s.neutralScore(),
                s.signal(), s.strength(), s.reasons()
            )
```

```

    );

    return chatModel.chat(prompt);
}
}

```

## 8. Kotlin - Gemma3 분석 Service

```

@Service
class XrpAiService(
    private val chatModel: ChatModel
) {
    fun analyze(i: IndicatorDto, s: SignalDto): String {

        val prompt = """
            당신은 XRP 기술적 분석 설명 AI입니다.

            [기술지표]
            현재가격: ${i.price}
            MA5: ${i.ma5}
            MA20: ${i.ma20}
            EMA12: ${i.ema12}
            EMA26: ${i.ema26}
            Wilder RSI14: ${i.rsi14}
            MACD: ${i.macd}
            Signal: ${i.macdSignal}
            Histogram: ${i.macdHistogram}
            Bollinger Upper: ${i.bollingerUpper}
            Bollinger Middle: ${i.bollingerMiddle}
            Bollinger Lower: ${i.bollingerLower}

            [Spring Boot 신호 계산]
            상승점수: ${s.bullishScore}
            하락점수: ${s.bearishScore}
            중립점수: ${s.neutralScore}
            판정: ${s.signal}
            강도: ${s.strength}
            계산 근거: ${s.reasons}

            계산 결과를 변경하지 말고 이해하기 쉽게 설명하세요.

            현재 추세:
            지표 해석:
            상승 근거:
            하락 위험:
            거래량 해석:
            종합 의견:
            주의사항:

            점수를 상승 확률로 표현하지 마세요.
            확정적인 투자 조언을 하지 마세요.
            """.trimIndent()

        return chatModel.chat(prompt)
    }
}

```

## 9. Controller 전체 파이프라인

---

### Java

```
@GetMapping("/predict")  
public Map<String, Object> predict() {  
  
    List<CandleDto> candles =  
        upbitService.getXrpCandles();  
  
    IndicatorDto indicators =  
        indicatorService.calculate(candles);  
  
    SignalDto signal =  
        signalService.calculate(indicators);  
  
    String analysis =  
        aiService.analyze(indicators, signal);  
  
    return Map.of(  
        "indicators", indicators,  
        "signal", signal,  
        "aiAnalysis", analysis  
    );  
}
```

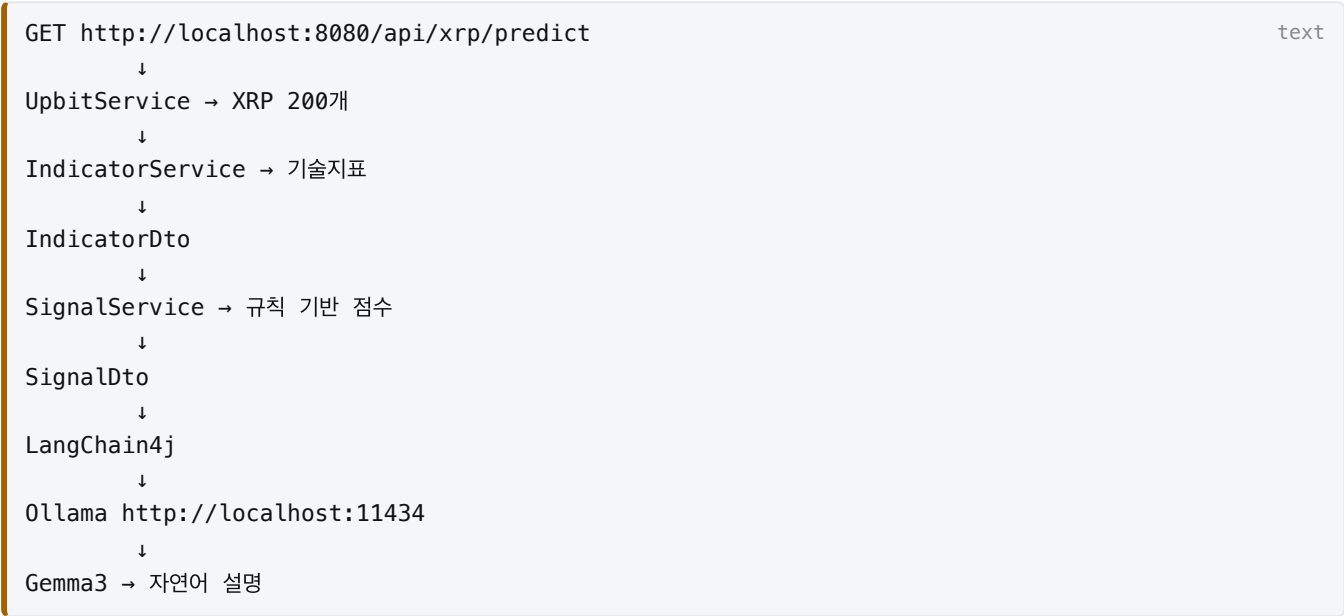
java

### Kotlin

```
@GetMapping("/predict")  
fun predict(): Map<String, Any> {  
  
    val candles =  
        upbitService.getXrpCandles()  
  
    val indicators =  
        indicatorService.calculate(candles)  
  
    val signal =  
        signalService.calculate(indicators)  
  
    val analysis =  
        aiService.analyze(indicators, signal)  
  
    return mapOf(  
        "indicators" to indicators,  
        "signal" to signal,  
        "aiAnalysis" to analysis  
    )  
}
```

kotlin

## 10. 최종 실행 흐름



## 11. Java ↔ Kotlin 핵심 비교

| 항목                 | Java         | Kotlin                |
|--------------------|--------------|-----------------------|
| DTO                | record       | data class            |
| 점수 변수              | int          | var                   |
| 조건 분기              | if / else if | if / when             |
| 이유 목록              | List<String> | mutableListOf<String> |
| 상한 제한              | Math.min     | coerceAtMost          |
| DTO 접근             | d.ma5()      | d.ma5                 |
| Spring 구조          | 동일           | 동일                    |
| LangChain4j/Gemma3 | 동일           | 동일                    |

## 12. 다음 발전 단계

다음 단계는 15분봉·60분봉·240분봉을 동시에 계산하는 MultiTimeframeService입니다. 각 시간대의 IndicatorDto와 SignalDto를 만든 뒤 단기·중기·큰 추세를 종합할 수 있습니다.

그 다음 MariaDB에 시점별 지표·점수·실제 이후 가격을 저장하면 백테스트를 통해 점수 규칙의 성능을 검증하고 가중치를 조정할 수 있습니다.

주의: 규칙 점수와 Gemma3 설명은 투자수익 또는 미래가격을 보장하지 않습니다.